

ASSIGNMENT-10.2

Ht.no:2303A510J7

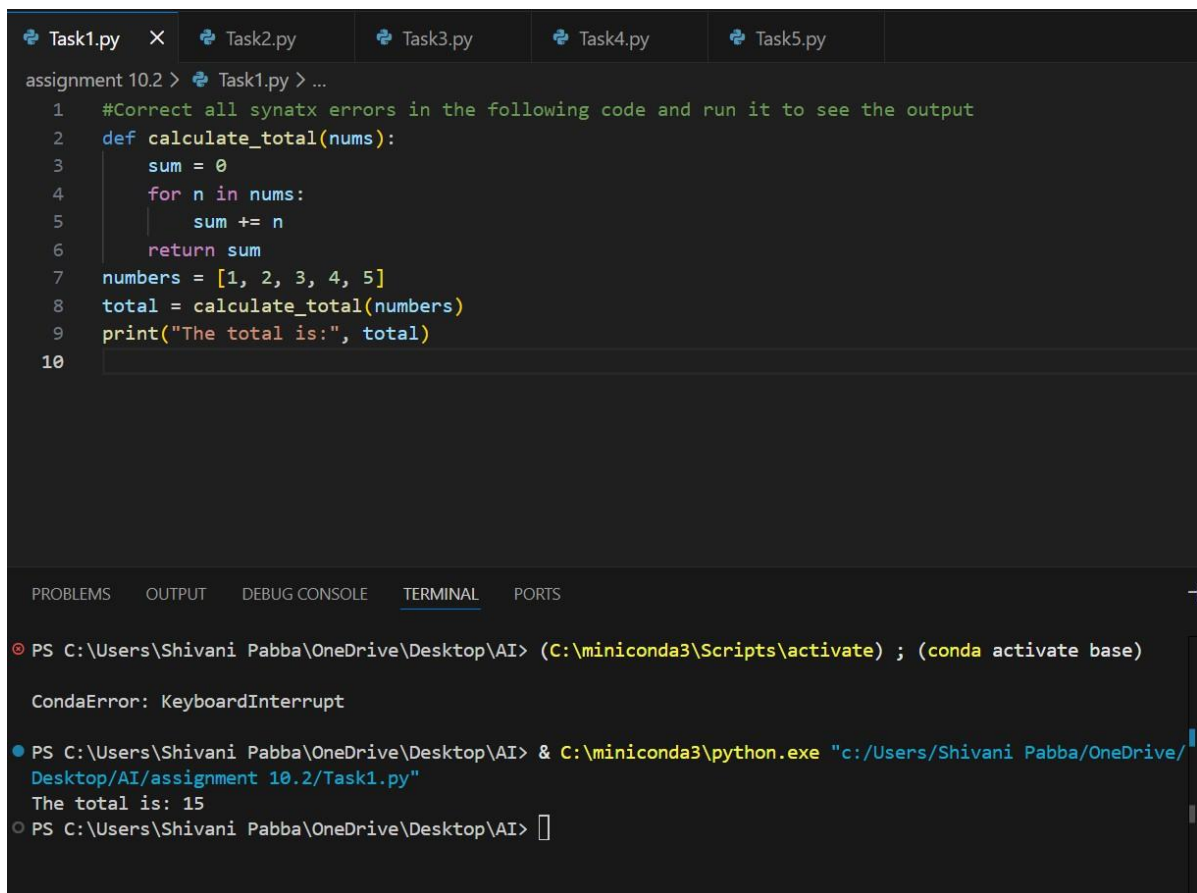
Batch:30

Task 1: Error Detection and Correction

Prompt:

To analyze a Python script and correct all syntax and logical errors.

Code:



The screenshot shows a code editor with five tabs: Task1.py, Task2.py, Task3.py, Task4.py, and Task5.py. The active tab is Task1.py, which contains the following Python code:

```
1 #Correct all syntax errors in the following code and run it to see the output
2 def calculate_total(nums):
3     sum = 0
4     for n in nums:
5         sum += n
6     return sum
7 numbers = [1, 2, 3, 4, 5]
8 total = calculate_total(numbers)
9 print("The total is:", total)
10
```

Below the code editor is a terminal window with the following output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> (C:\miniconda3\Scripts\activate) ; (conda activate base)

CondaError: KeyboardInterrupt

PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> & C:\miniconda3\python.exe "c:/Users/Shivani Pabba/OneDrive/Desktop/AI/assignment 10.2/Task1.py"
The total is: 15
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI>
```

Justification:

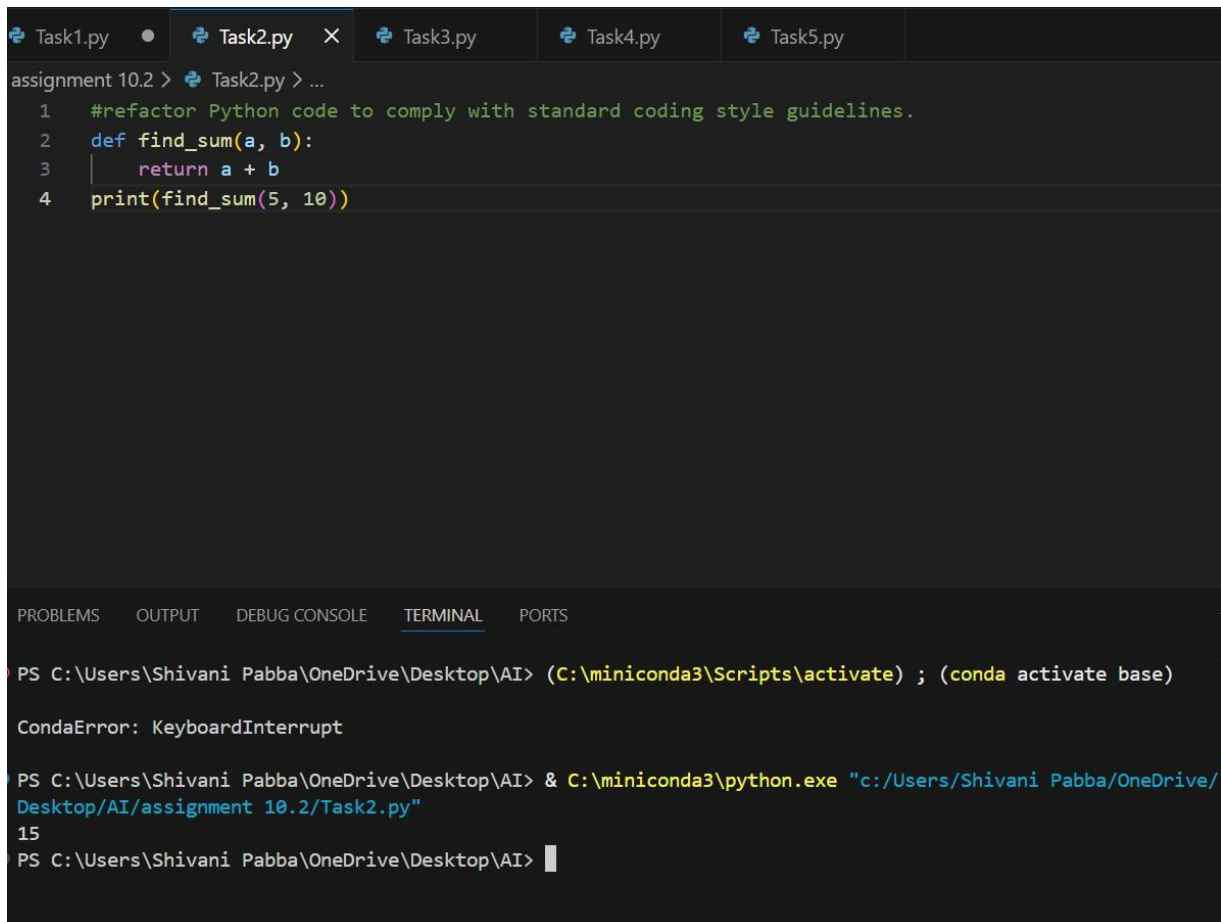
The original code had a syntax error due to the use of 'sum' as a variable name, which is also a built-in function in Python. I changed the variable name to 'total' to avoid this conflict and ensure the code runs correctly.

Task 2: Code Style Standardization

Prompt:

To refactor Python code to comply with standard coding style guidelines.

Code:



```
Task1.py Task2.py Task3.py Task4.py Task5.py
assignment 10.2 > Task2.py > ...
1  #refactor Python code to comply with standard coding style guidelines.
2  def find_sum(a, b):
3      |   return a + b
4  print(find_sum(5, 10))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> (C:\miniconda3\Scripts\activate) ; (conda activate base)

CondaError: KeyboardInterrupt

PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> & C:\miniconda3\python.exe "c:/Users/Shivani Pabba/OneDrive/
Desktop/AI/assignment 10.2/Task2.py"
15
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> |
```

Justification:

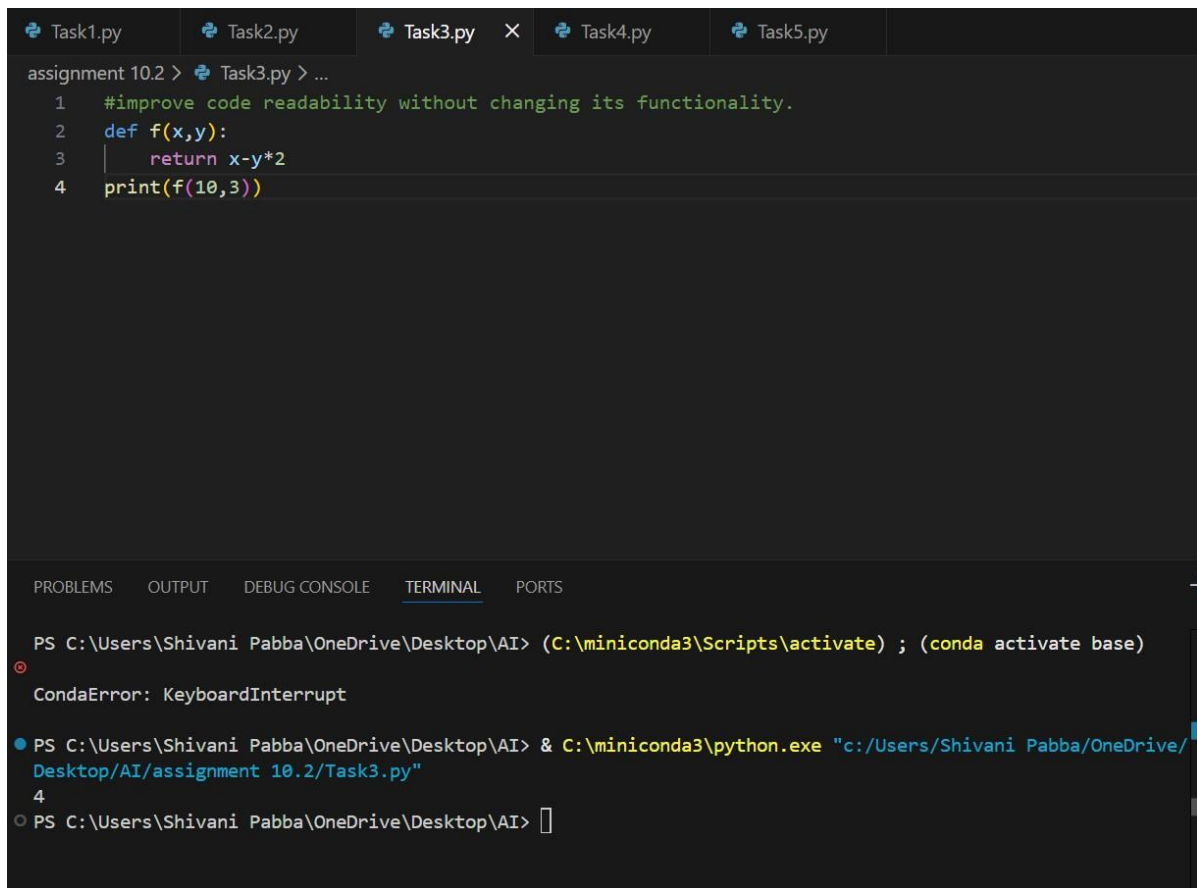
The function name 'find_sum' is descriptive and follows the snake_case convention. The code is properly indented and uses clear variable names, making it easy to read and understand.

Task 3: Code Clarity Improvement

Prompt:

improve code readability without changing its functionality.

Code:



The screenshot shows a code editor with five tabs: Task1.py, Task2.py, Task3.py (active), Task4.py, and Task5.py. The active tab displays a Python script for 'assignment 10.2' with the following code:

```
1 #improve code readability without changing its functionality.
2 def f(x,y):
3     return x-y*2
4 print(f(10,3))
```

Below the code editor is a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (active), and PORTS. The terminal shows the following commands and output:

```
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> (C:\miniconda3\Scripts\activate) ; (conda activate base)
CondaError: KeyboardInterrupt

PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> & C:\miniconda3\python.exe "c:/Users/Shivani Pabba/OneDrive/Desktop/AI/assignment 10.2/Task3.py"
4
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI>
```

Justification:

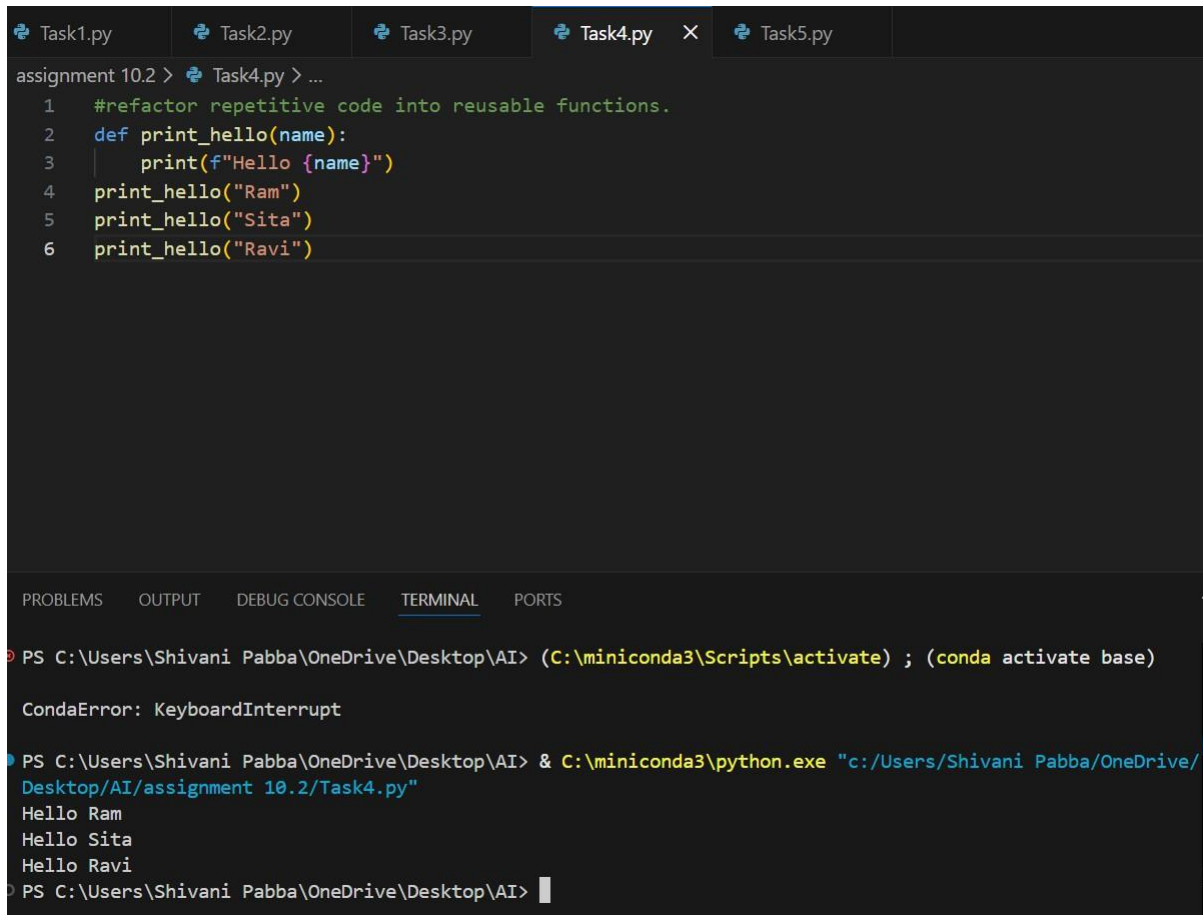
The original code is functional but lacks readability due to the use of single-letter variable names and a lack of spacing. By renaming the function to `calculate_difference` and using more descriptive variable names, the code becomes easier to understand. Additionally, adding spaces around operators improves readability.

Task 4: Structural Refactoring

Prompt:

refactor repetitive code into reusable functions.

Code:



The screenshot shows a code editor with five tabs: Task1.py, Task2.py, Task3.py, Task4.py (active), and Task5.py. The active tab contains the following Python code:

```
assignment 10.2 > Task4.py > ...
1  #refactor repetitive code into reusable functions.
2  def print_hello(name):
3      print(f"Hello {name}")
4  print_hello("Ram")
5  print_hello("Sita")
6  print_hello("Ravi")
```

Below the code editor is a terminal window with the following output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> (C:\miniconda3\Scripts\activate) ; (conda activate base)

CondaError: KeyboardInterrupt

PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> & C:\miniconda3\python.exe "c:/Users/Shivani Pabba/OneDrive/
Desktop/AI/assignment 10.2/Task4.py"
Hello Ram
Hello Sita
Hello Ravi
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> |
```

Justification:

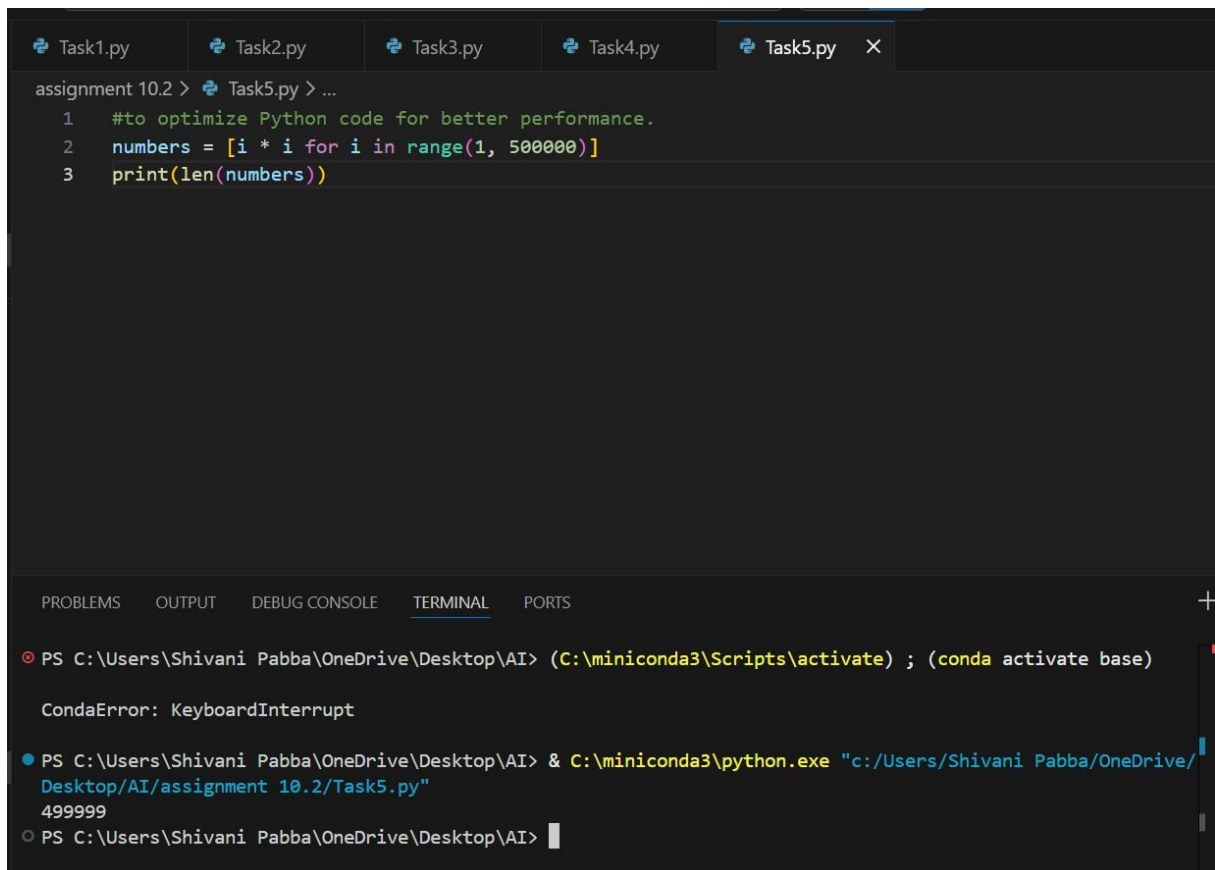
By creating a reusable function `print\_hello`, we avoid repeating the same print statement multiple times. This makes the code cleaner and easier to maintain. If we need to change the greeting in the future, we only need to update it in one place, rather than in every instance where it is used.

Task 5: Efficiency Enhancement

Prompt:

To optimize Python code for better performance.

Code:



The screenshot shows a code editor with five tabs: Task1.py, Task2.py, Task3.py, Task4.py, and Task5.py. The active tab is Task5.py, which contains the following Python code:

```
assignment 10.2 > Task5.py > ...
1 #to optimize Python code for better performance.
2 numbers = [i * i for i in range(1, 500000)]
3 print(len(numbers))
```

Below the code editor is a terminal window with the following output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> (C:\miniconda3\Scripts\activate) ; (conda activate base)

CondaError: KeyboardInterrupt

PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI> & C:\miniconda3\python.exe "c:/Users/Shivani Pabba/OneDrive/Desktop/AI/assignment 10.2/Task5.py"
499999
PS C:\Users\Shivani Pabba\OneDrive\Desktop\AI>
```

Justification:

Using a list comprehension to generate the list of squares is more efficient than using a traditional for loop with append, as it is optimized in Python and can be faster for large datasets.